

**Universidad Autónoma de Tamaulipas  
Facultad de Ingeniería Tampico**



**Proyecto Analizador Léxico  
ExcelToJava**

**Carrera: Ingeniería en Sistemas Computacionales**

**Materia: Programación de Sistemas de Base 1**

**Profesor: Muñoz Quintero Dante Adolfo**

**Equipo:**

**Díaz Villarreal Celine Michelle**

**Martínez Acuña Brandon**

**Ibarra Loredó Juan Jesús**

**Cabrera Reyes Carlos Alberto**

**Semestre: 8°K**

**Fecha: 16 de mayo del 2026**

## Índice

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Índice .....                                                                             | 2  |
| 1. Introducción .....                                                                    | 3  |
| 2. Objetivos .....                                                                       | 4  |
| 3. Especificación del Lenguaje .....                                                     | 5  |
| 3.1 Nombre del Lenguaje y Justificación. ....                                            | 5  |
| 3.2 Descripción de la Sintaxis .....                                                     | 5  |
| 3.3 Tabla de Tokens .....                                                                | 7  |
| 4. Implementación .....                                                                  | 8  |
| 4.1 Definición de Tokens y Palabras Reservadas .....                                     | 8  |
| 4.2 Reglas Léxicas .....                                                                 | 10 |
| 4.3 Lógica del Analizador Léxico .....                                                   | 13 |
| 4.4 Manejo y Reporte de Errores .....                                                    | 17 |
| 5. Demostración Funcional .....                                                          | 18 |
| 5.1 Ejecución con un programa valido y su lista de Tokens generada .....                 | 18 |
| 5.2 Ejecución con entradas que contienen errores léxicos y los mensajes producidos ..... | 21 |
| 5.3 Identificadores Inválidos y/o símbolos desconocidos .....                            | 22 |
| 6. Conclusiones .....                                                                    | 23 |
| Aprendizajes Individuales .....                                                          | 23 |
| Dificultades encontradas durante el desarrollo .....                                     | 24 |
| Trabajo Futuro .....                                                                     | 25 |
| Posibles Mejoras .....                                                                   | 25 |
| 7. Referencias .....                                                                     | 25 |

## 1. Introducción

Microsoft Excel es una de las herramientas de cálculo más utilizadas en el mundo académico y empresarial. Sus funciones matemáticas y lógicas, como SUMA, POTENCIA o SI, permiten a usuarios no programadores expresar cálculos complejos de forma concisa y directa. Sin embargo, cuando estos cálculos necesitan integrarse en un sistema de software desarrollado en Java, el programador debe reescribir manualmente cada fórmula en sintaxis Java, lo que implica conocer equivalencias como equivalencias, operadores relacionales ( $\neq$ ,  $==$ ) y el manejo de colecciones para rangos de celdas.

### **Problema o dominio que busca resolver**

El proyecto busca dar un primer paso hacia un traductor automático de fórmulas Excel a código Java, comenzando por la fase de análisis léxico. El dominio del problema es la brecha de sintaxis entre el lenguaje de fórmulas de Excel y el lenguaje de programación Java:

- Excel usa  $=$  para comparar igualdad; Java usa  $==$ .
- Excel usa  $<>$  para "diferente de"; Java usa  $\neq$ .
- Excel por ejemplo usa POTENCIA (base, exp) mientras que Java usa  $\text{Math.pow}(\text{base}, \text{exp})$ .
- Excel referencia datos mediante celdas (A1, B5) y rangos (A1:B5); Java no tiene ese concepto nativo.

El analizador léxico desarrollado en este proyecto identifica y clasifica cada elemento de una fórmula Excel (funciones, operadores, referencias a celdas, literales, etc.) en tokens con significado semántico, sentando las bases para las fases posteriores de análisis sintáctico, semántico y generación de código Java.

## 2. Objetivos

### Objetivo General

Diseñar e implementar un analizador léxico funcional para el lenguaje **ExcelToJava**, capaz de transformar expresiones matemáticas y lógicas escritas en sintaxis Excel en una lista ordenada de tokens, como primer paso de un proceso de traducción a código Java.

### Objetivos Específicos

1. **Especificación del Lenguaje:** Definir formalmente el conjunto de tokens que componen el lenguaje ExcelToJava, estableciendo las categorías, patrones de reconocimiento y la semántica de cada elemento léxico.
2. **Implementación del Analizador Léxico:** Construir un lexer funcional utilizando la herramienta ANTLR4 como motor de reconocimiento, integrado con una capa propia en Python que encapsula la lógica de conversión y traducción de tokens.
3. **Pruebas y Validación con Entradas Externas:** Verificar el correcto funcionamiento del lexer mediante archivos de prueba .txt, tanto con expresiones válidas (que deben producir una lista limpia de tokens) como con entradas inválidas (que deben reportar errores léxicos con número de línea y columna precisos).
4. **Manejo y Reporte de Errores Léxicos:** Garantizar que el analizador no se detenga ante errores, sino que los acumule y reporte de forma descriptiva, identificando la posición exacta del símbolo problemático.

## 3. Especificación del Lenguaje

### 3.1 Nombre del Lenguaje y Justificación.

Nombre: **ExcelToJava**

Justificación: El nombre refleja directamente el propósito del lenguaje: tomar como entrada expresiones escritas en la sintaxis de funciones de Microsoft Excel y producir como salida su equivalente en Java.

### 3.2 Descripción de la Sintaxis

El lenguaje ExcelToJava acepta **expresiones matemáticas y lógicas** escritas con la misma sintaxis que se usa en las celdas de Microsoft Excel. La especificación del lenguaje se encuentra en el archivo **grammar/ExcelLexer.g4**.

A continuación, se ilustran los patrones principales con ejemplos de código:

#### Funciones

Las funciones siguen el patrón **NOMBRE(argumento1, argumento2, ...)**. Los nombres de función son palabras reservadas en mayúsculas.

`SUMA(A1:B5)`

`POTENCIA(A1, 2)`

`SI(A1 <> 10, SUMA(B1:B5), 0)`

`PROMEDIO($A$1:$C$3)`

`CONCATENAR("Hola", " ", "Mundo")`

#### Referencias a Celdas

Las referencias pueden ser relativas (A1, B10) o absolutas (\$A\$1, \$C\$3). Los rangos conectan dos referencias con:

A1               # Referencia relativa simple

\$B\$2           # Referencia absoluta

A1:B5          # Rango relativo

\$A\$1:\$C\$3     # Rango absoluto

#### Operadores

Los operadores aritméticos y relacionales se usan directamente dentro de expresiones o como argumentos de funciones lógicas (SI, Y, O).

$A1 + B1$

$A1 * 3.14$

$A1 \geq 10$

$A1 \neq 0$  # Diferente de (se traducirá a  $\neq$ )

$A1 = B1$  # Igualdad (se traducirá a  $==$ )

### Literales Numéricos y de Texto

42 # Entero

3.14 # Flotante

"Hola Mundo" # Cadena de texto (entre comillas dobles)

### Funciones Lógicas

Y, O y NO solo se reconocen como funciones cuando van seguidas de (. De lo contrario, se tratan como identificadores genéricos.

$Y(A1 > 0, B1 < 10)$  # Y = función lógica AND

$O(A1 = 1, B1 = 2)$  # O = función lógica OR

$NO(A1 > 5)$  # NO = función lógica NOT

Y # Y sin paréntesis = IDENTIFIER

### Separadores Alternativos

El lenguaje acepta tanto coma (,) como punto y coma (;) como separadores de argumentos.

SUMA(A1, B1, C1) # Separador coma

SUMA(A1; B1; C1) # Separador punto y coma

### 3.3 Tabla de Tokens

| <i>Categoría</i>  | <i>Descripción</i>                                          | <i>Patrón / Expresión Regular (ANTLR)</i>                                                                                                                   |
|-------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>FUNCTION</i>   | Función reconocida (palabra reservada)                      | 'SUMA'   'PROMEDIO'   'MAX'   'MIN'   'CONTAR'   'CONCATENAR'   'POTENCIA'   'RAIZ'   'ABS'   'REDONDEAR'   'ENTERO'   'SI'   ('Y' 'O' 'NO') seguido de '(' |
| <i>CELL_RANGE</i> | Rango de celdas (relativo o absoluto)                       | (''\$'? [A-Z]+ '\$'? [0-9]+) ':' (''\$'? [A-Z]+ '\$'? [0-9]+)                                                                                               |
| <i>CELL_REF</i>   | Referencia a celda individual (relativa o absoluta)         | '\$'? [A-Z]+ '\$'? [0-9]+                                                                                                                                   |
| <i>FLOAT</i>      | Número de punto flotante                                    | [0-9]+ '.' [0-9]+                                                                                                                                           |
| <i>INTEGER</i>    | Número entero                                               | [0-9]+                                                                                                                                                      |
| <i>STRING</i>     | Cadena de texto entre comillas dobles                       | """ (~["\r\n])* """                                                                                                                                         |
| <i>IDENTIFIER</i> | Identificador genérico (letras, dígitos, guión bajo)        | [A-Za-z_][A-Za-z0-9_]*                                                                                                                                      |
| <i>OPERATOR</i>   | Operador aritmético o relacional                            | '<='   '>='   '<>'   '+'   '-'   '*'   '/'   '^'   '<'   '>'   '='                                                                                          |
| <i>LPAREN</i>     | Paréntesis de apertura                                      | '('                                                                                                                                                         |
| <i>RPAREN</i>     | Paréntesis de cierre                                        | )'                                                                                                                                                          |
| <i>COMMA</i>      | Coma — separador de argumentos                              | ','                                                                                                                                                         |
| <i>SEMICOLON</i>  | Punto y coma — separador alternativo de argumentos          | ';'                                                                                                                                                         |
| <i>EOF</i>        | Fin de entrada                                              | (generado automáticamente por ANTLR)                                                                                                                        |
| <i>ERROR</i>      | Símbolo desconocido o identificador inválido (error léxico) | . (catch-all: cualquier carácter no reconocido por ninguna regla)                                                                                           |

**Nota de prioridad:** Las reglas están ordenadas por especificidad. CELL\_RANGE precede a CELL\_REF para que A1:B5 no se tokenice como tres piezas. FLOAT precede a INTEGER para que 3.14 no se tokenice como 3, ., 14. Los operadores de dos caracteres (<=, >=, <>) preceden a los de uno.

## 4. Implementación

### 4.1 Definición de Tokens y Palabras Reservadas.

**Archivo:** src/tokens.py

```
from __future__ import annotations

from enum import Enum, auto
from typing import Any

# Enumeracion con los 14 tipos de token del lenguaje
class TokenType(Enum):
    # Funciones Excel
    FUNCTION = auto() # SUMA, PROMEDIO, SI, POTENCIA, ...

    # Referencias a celdas
    CELL_RANGE = auto() # A1:B5, $A$1:$C$3
    CELL_REF = auto() # A1, $B$2

    # Literales numéricos
    FLOAT = auto() # 3.14
    INTEGER = auto() # 42

    # Literal de texto
    STRING = auto() # "Hola"

    # Identificadores genéricos
    IDENTIFIER = auto() # nombre_variable, Y (sin paréntesis)

    # Operadores
    OPERATOR = auto() # +, -, *, /, ^, <, >, <=, >=, <>, =

    # Delimitadores
    LPAREN = auto() # (
    RPAREN = auto() # )
```





```
COMMA      = auto()      # ,
SEMICOLON  = auto()      # ;

# Control
EOF        = auto()
ERROR      = auto()

# Token – Unidad léxica individual
class Token:

    # Constructor
    def __init__(self, type: TokenType, value: Any,
                  line: int = 0, column: int = 0) -> None:
        self.type: TokenType = type
        self.value: Any      = value
        self.line: int       = line
        self.column: int     = column

    # Formatea el token para impresion en pantalla
    def __str__(self) -> str:
        return (
            f"({self.type.name} , "
            f"{self.value!r} , "
            f"línea={self.line}, col={self.column})"
        )

    def __repr__(self) -> str:
        return self.__str__()

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Token):
            return False
        return self.type == other.type and self.value == other.value

    # Consultas de tipo

    def isType(self, token_type: TokenType) -> bool:
        return self.type == token_type

    def isFunction(self) -> bool: return self.isType(TokenType.FUNCTION)
    def isCellRange(self) -> bool: return self.isType(TokenType.CELL_RANGE)
    def isCellRef(self) -> bool: return self.isType(TokenType.CELL_REF)
    def isFloat(self) -> bool: return self.isType(TokenType.FLOAT)
    def isInteger(self) -> bool: return self.isType(TokenType.INTEGER)
```

```
def isString(self)      -> bool: return self.isType(TokenType.STRING)
def isIdentifier(self)  -> bool: return self.isType(TokenType.IDENTIFIER)
def isOperator(self)   -> bool: return self.isType(TokenType.OPERATOR)
def isLParen(self)     -> bool: return self.isType(TokenType.LPAREN)
def isRParen(self)     -> bool: return self.isType(TokenType.RPAREN)
def isComma(self)      -> bool: return self.isType(TokenType.COMMA)
def isSemicolon(self)  -> bool: return self.isType(TokenType.SEMICOLON)
def isEOF(self)        -> bool: return self.isType(TokenType.EOF)
def isError(self)      -> bool: return self.isType(TokenType.ERROR)

def has_value(self, expected: Any) -> bool:
    return self.value == expected

# Checa si el token es de cierto tipo y tiene un valor
def matches(self, token_type: TokenType, value: Any = None) -> bool:
    if value is None:
        return self.isType(token_type)
    return self.isType(token_type) and self.has_value(value)
```

Las **palabras reservadas** (funciones Excel) no se definen en tokens.py sino directamente en la gramática ANTLR (grammar/ExcelLexer.g4, regla FUNCTION, líneas 9–23), ya que ANTLR las reconoce antes que la regla IDENTIFIER.

## 4.2 Reglas Léxicas

**Archivo:** grammar/ExcelLexer.g4

lexer grammar ExcelLexer;

```
// =====
// FUNCIONES EXCEL
// Regla de prioridad: reglas más específicas primero.
// Y, O, NO solo son FUNCTION cuando van seguidas de '('.
// =====
```

```
FUNCTION
    : 'SUMA'
    | 'PROMEDIO'
    | 'MAX'
    | 'MIN'
```

```
| 'CONTAR '  
| 'CONCATENAR '  
| 'POTENCIA '  
| 'RAIZ '  
| 'ABS '  
| 'REDONDEAR '  
| 'ENTERO '  
| 'SI '  
| ('Y' | 'O' | 'NO') { self._input.LA(1) == ord('(') }?  
;
```

```
// =====  
// REFERENCIAS A CELDAS  
// CELL_RANGE DEBE ir antes que CELL_REF (maximal munch de ANTLR).  
// Soporta referencias absolutas ($A$1) y relativas (A1).  
// =====
```

```
fragment COL_PART : '$'? [A-Z]+ ;  
fragment ROW_PART : '$'? [0-9]+ ;  
fragment CELL_ID : COL_PART ROW_PART ;
```

```
CELL_RANGE : CELL_ID ':' CELL_ID ;  
CELL_REF : CELL_ID ;
```

```
// =====  
// LITERALES NUMÉRICOS  
// FLOAT DEBE ir antes que INTEGER.  
// =====
```

```
FLOAT : [0-9]+ '.' [0-9]+ ;  
INTEGER : [0-9]+ ;
```

```
// =====
```

// CADENAS DE TEXTO

// =====

STRING : ''' (~["\r\n])\* ''' ;

// =====

// OPERADORES

// Los operadores de DOS caracteres DEBEN ir antes que los simples.

// Traducciones aplicadas en Lexer.py: <> → != | = → ==

// =====

OPERATOR

```
: '<='      // Menor o igual
| '>='      // Mayor o igual
| '<>'      // Diferente (Excel) → != (Java)
| '+'
| '-'
| '*'
| '/'
| '^'      // Potencia (Excel) → Math.pow() (Java)
| '<'
| '>'
| '='      // Igual (Excel) → == (Java)
;
```

// =====

// DELIMITADORES

// =====

LPAREN : '(' ;

RPAREN : ')' ;

COMMA : ',' ;

SEMICOLON : ';' ;

```
// =====  
// IDENTIFICADORES GENÉRICOS  
// Letras + dígitos + guión bajo; NO puede empezar con dígito.  
// =====
```

```
IDENTIFIER : [A-Za-z_][A-Za-z0-9_]* ;
```

```
// =====  
// ESPACIOS EN BLANCO → ignorados  
// =====
```

```
WS : [ \t\r\n]+ -> skip ;
```

```
// =====  
// CATCH-ALL DE ERRORES  
// Todo carácter no reconocido genera token UNKNOWN.  
// =====
```

```
UNKNOWN : . ;
```

## 4.3 Lógica del Analizador Léxico

**Archivo:** src/lexer.py

```
from __future__ import annotations  
  
import os  
import sys  
from typing import List  
  
# Añadir la raíz del proyecto al path  
_PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))  
if _PROJECT_ROOT not in sys.path:  
    sys.path.insert(0, _PROJECT_ROOT)  
  
import antlr4
```

```
from grammar.generated.ExcelLexer import ExcelLexer as ANTLRExcelLexer

from tokens import Token, TokenType
from errors import LexerErrorListener

# Traducción de operadores Excel a Java
OPERATOR_TRANSLATIONS: dict[str, str] = {
    '<>': '!=',
    '=': '==',
}

# Diccionario de traducción de tipos de ANTLR a TokenType
ANTLR_TO_TOKEN_TYPE: dict[str, TokenType] = {
    'FUNCTION': TokenType.FUNCTION,
    'CELL_RANGE': TokenType.CELL_RANGE,
    'CELL_REF': TokenType.CELL_REF,
    'FLOAT': TokenType.FLOAT,
    'INTEGER': TokenType.INTEGER,
    'STRING': TokenType.STRING,
    'IDENTIFIER': TokenType.IDENTIFIER,
    'OPERATOR': TokenType.OPERATOR,
    'LPAREN': TokenType.LPAREN,
    'RPAREN': TokenType.RPAREN,
    'COMMA': TokenType.COMMA,
    'SEMICOLON': TokenType.SEMICOLON,
    'UNKNOWN': TokenType.ERROR,
}

# Clase Lexer que convierte la fuente en tokens
class Lexer:

    # Constructor
    def __init__(self, fuente: str) -> None:
        self.fuente: str = fuente
        self.tokens: List[Token] = []
        self.errores: List[str] = []

    def scan_tokens(self) -> List[Token]:
        # Tokeniza la fuente y retorna la lista de Token.
        input_stream = antlr4.InputStream(self.fuente)
        antlr_lexer = ANTLRExcelLexer(input_stream)
```

```
# Listener personalizado para errores léxicos
error_listener = LexerErrorListener()
antlr_lexer.removeErrorListeners()
antlr_lexer.addErrorListener(error_listener)

stream = antlr4.CommonTokenStream(antlr_lexer)
stream.fill()
self.errores = error_listener.errores

# Chequeo de adyacencia: detectar INTEGER/FLOAT pegado a IDENTIFIER
raw = [t for t in stream.tokens if t.type != antlr4.Token.EOF]
for i in range(len(raw) - 1):
    curr, nxt = raw[i], raw[i + 1]
    curr_sym = (antlr_lexer.symbolicNames[curr.type]
                 if 0 < curr.type < len(antlr_lexer.symbolicNames) else '')
    nxt_sym = (antlr_lexer.symbolicNames[nxt.type]
                if 0 < nxt.type < len(antlr_lexer.symbolicNames) else '')
    if curr_sym in ('INTEGER', 'FLOAT') and nxt_sym == 'IDENTIFIER':
        # Son adyacentes si el IDENTIFIER empieza justo donde termina el
número
        if nxt.line == curr.line and nxt.column == curr.column +
len(curr.text):
            self.errores.append(
                f"Error léxico - línea {curr.line}, columna {curr.column +
1}: "

                f"identificador inválido '{curr.text}{nxt.text}' "
                f"(los identificadores no pueden comenzar con un dígito)"
            )

for antlr_token in stream.tokens:
    token = self._convertir(antlr_token, antlr_lexer)
    if token is not None:
        self.tokens.append(token)

return self.tokens

def _convertir(self, antlr_token, antlr_lexer) -> Token | None:
    # Convierte un antlr4.Token a nuestro Token interno.
    ttype = antlr_token.type
    text = antlr_token.text
    linea = antlr_token.line
    col = antlr_token.column + 1 # ANTLR usa índice 0, lo convierte a 1

    # EOF
    if ttype == antlr4.Token.EOF:
```

```
        return Token(TokenType.EOF, '', linea, col)

    # Nombre simbólico del tipo (ej. 'FUNCTION', 'OPERATOR', ...)
    sym_name = (
        antlr_lexer.symbolicNames[ttype]
        if 0 < ttype < len(antlr_lexer.symbolicNames)
        else 'UNKNOWN'
    )

    our_type = ANTLR_TO_TOKEN_TYPE.get(sym_name, TokenType.ERROR)

    # Transformaciones de valor
    value = text

    if our_type == TokenType.OPERATOR:
        value = OPERATOR_TRANSLATIONS.get(text, text)

    elif our_type == TokenType.STRING:
        value = text[1:-1]          # Quitar comillas

    elif our_type == TokenType.FLOAT:
        value = float(text)

    elif our_type == TokenType.INTEGER:
        value = int(text)

    # Token desconocido, registra error adicional con contexto
    if sym_name == 'UNKNOWN':
        selferrores.append(
            f"Error léxico - línea {linea}, columna {col}: "
            f"símbolo desconocido '{text}'"
        )
        return Token(TokenType.ERROR, text, linea, col)
    return Token(our_type, value, linea, col)

def print_tokens(self) -> None:
    # Imprime la lista de tokens y errores
    validos = [t for t in self.tokens if not t.isEOF()]

    print(f" Tokens reconocidos : {len(validos)}")
    for i, token in enumerate(validos, 1):
        print(f"  {i:>3}: {token}")

    if self.errores:
```



```
print(" Errores léxicos :")
for err in self.errores:
    print(f" {err}")
```

## 4.4 Manejo y Reporte de Errores

Archivo: src/errors.py

```
# Contiene el listener de errores que ANTLR4 invoca cuando encuentra un símbolo no
reconocido

from typing import List

from antlr4.error.ErrorListener import ErrorListener

class LexerErrorListener(ErrorListener):

    # Intercepta los errores léxicos de ANTLR4 sin detener el análisis.

    def __init__(self) -> None:
        super().__init__()
        self.errores: List[str] = []

    # Metodo que es invocado por ANTLR4 cuando encuentra un error léxico
    def syntaxError(self, recognizer, offendingSymbol,
                    line: int, column: int, msg: str, e) -> None:
        self.errores.append(
            f"Error léxico - línea {line}, columna {column + 1}: {msg}"
        )
```

Detección adicional dentro de scan\_tokens en leer.py.

```
# Chequeo de adyacencia: detectar INTEGER/FLOAT pegado a IDENTIFIER
raw = [t for t in stream.tokens if t.type != antlr4.Token.EOF]
for i in range(len(raw) - 1):
    curr, nxt = raw[i], raw[i + 1]
    curr_sym = (antlr_lexer.symbolicNames[curr.type]
                if 0 < curr.type < len(antlr_lexer.symbolicNames) else '')
    nxt_sym = (antlr_lexer.symbolicNames[nxt.type]
               if 0 < nxt.type < len(antlr_lexer.symbolicNames) else '')
    if curr_sym in ('INTEGER', 'FLOAT') and nxt_sym == 'IDENTIFIER':
        # Son adyacentes si el IDENTIFIER empieza justo donde termina el
número
```

```

        if nxt.line == curr.line and nxt.column == curr.column +
len(curr.text):
            self.errores.append(
                f"Error léxico — línea {curr.line}, columna {curr.column +
1}: "

                f"identificador inválido '{curr.text}{nxt.text}' "
                f"(los identificadores no pueden comenzar con un dígito)"
            )

```

| Tipo de error                                     | Ejemplo de entrada | Mensaje generado                                                                                                          |
|---------------------------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Símbolo desconocido</b>                        | @variable          | Error léxico — línea 1, columna 1: símbolo desconocido '@'                                                                |
| <b>Identificador inválido (inicia con dígito)</b> | SUMA(123abc)       | Error léxico — línea 1, columna 6: identificador inválido '123abc' (los identificadores no pueden comenzar con un dígito) |

## 5. Demostración Funcional

### 5.1 Ejecución con un programa valido y su lista de Tokens generada

Archivo de entrada – *tests/valid/test01.txt*

# test01.txt – Entradas válidas: funciones básicas y referencias a celdas  
# Cada línea es una expresión independiente.

```

SUMA(A1:B5)
SI(A1<>10, SUMA(B1:B5), 0)
POTENCIA(A1, 2)
Y(A1>0, B1<10)
O(A1=1, B1=2)
NO(A1>5)

```

## Salida y Ejecución:

```
=====
Analizador Léxico – ExcelToJava
Fórmulas Excel → código Java
=====
```

```
=====
Categoría: VALID
=====
```

```
=====
Archivo : test01.txt
=====
```

```
Línea  4: SUMA(A1:B5)
```

```
Tokens reconocidos : 4
```

- 1: (FUNCTION , 'SUMA' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=5)
- 3: (CELL\_RANGE , 'A1:B5' , línea=1, col=6)
- 4: (RPAREN , ')' , línea=1, col=11)

```
Línea  5: SI(A1<>10, SUMA(B1:B5), 0)
```

```
Tokens reconocidos : 13
```

- 1: (FUNCTION , 'SI' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=3)
- 3: (CELL\_REF , 'A1' , línea=1, col=4)
- 4: (OPERATOR , '!=' , línea=1, col=6)
- 5: (INTEGER , 10 , línea=1, col=8)
- 6: (COMMA , ',' , línea=1, col=10)
- 7: (FUNCTION , 'SUMA' , línea=1, col=12)
- 8: (LPAREN , '(' , línea=1, col=16)
- 9: (CELL\_RANGE , 'B1:B5' , línea=1, col=17)
- 10: (RPAREN , ')' , línea=1, col=22)
- 11: (COMMA , ',' , línea=1, col=23)
- 12: (INTEGER , 0 , línea=1, col=25)
- 13: (RPAREN , ')' , línea=1, col=26)

Línea 6: POTENCIA(A1, 2)

Tokens reconocidos : 6

- 1: (FUNCTION , 'POTENCIA' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=9)
- 3: (CELL\_REF , 'A1' , línea=1, col=10)
- 4: (COMMA , ',' , línea=1, col=12)
- 5: (INTEGER , 2 , línea=1, col=14)
- 6: (RPAREN , ')' , línea=1, col=15)

Línea 7: Y(A1>0, B1<10)

Tokens reconocidos : 10

- 1: (FUNCTION , 'Y' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=2)
- 3: (CELL\_REF , 'A1' , línea=1, col=3)
- 4: (OPERATOR , '>' , línea=1, col=5)
- 5: (INTEGER , 0 , línea=1, col=6)
- 6: (COMMA , ',' , línea=1, col=7)
- 7: (CELL\_REF , 'B1' , línea=1, col=9)
- 8: (OPERATOR , '<' , línea=1, col=11)
- 9: (INTEGER , 10 , línea=1, col=12)
- 10: (RPAREN , ')' , línea=1, col=14)

Línea 8: O(A1=1, B1=2)

Tokens reconocidos : 10

- 1: (FUNCTION , 'O' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=2)
- 3: (CELL\_REF , 'A1' , línea=1, col=3)
- 4: (OPERATOR , '==' , línea=1, col=5)
- 5: (INTEGER , 1 , línea=1, col=6)
- 6: (COMMA , ',' , línea=1, col=7)
- 7: (CELL\_REF , 'B1' , línea=1, col=9)
- 8: (OPERATOR , '==' , línea=1, col=11)
- 9: (INTEGER , 2 , línea=1, col=12)
- 10: (RPAREN , ')' , línea=1, col=13)

```
Línea 9: NO(A1>5)
Tokens reconocidos : 6
1: (FUNCTION , 'NO' , línea=1, col=1)
2: (LPAREN , '(' , línea=1, col=3)
3: (CELL_REF , 'A1' , línea=1, col=4)
4: (OPERATOR , '>' , línea=1, col=6)
5: (INTEGER , 5 , línea=1, col=7)
6: (RPAREN , ')' , línea=1, col=8)
```

```
Expresiones analizadas : 6
```

## 5.2 Ejecución con entradas que contienen errores léxicos y los mensajes producidos

**Archivo de entrada – tests/invalid/errores01.txt**

```
# errores01.txt – Entradas con errores léxicos esperados
# Cada línea es una expresión independiente.
# El lexer debe detectar y reportar los errores sin detenerse.
```

```
@variable
SUMA(123abc)
```

**Salida y Ejecución.**

```
=====
Categoría: INVALID
=====

Archivo : errores01.txt
=====

Línea 5: @variable
Tokens reconocidos : 2
1: (ERROR , '@' , línea=1, col=1)
2: (IDENTIFIER , 'variable' , línea=1, col=2)
Errores léxicos :
Error léxico – línea 1, columna 1: símbolo desconocido '@'

Línea 6: SUMA(123abc)
Tokens reconocidos : 5
1: (FUNCTION , 'SUMA' , línea=1, col=1)
2: (LPAREN , '(' , línea=1, col=5)
3: (INTEGER , 123 , línea=1, col=6)
4: (IDENTIFIER , 'abc' , línea=1, col=9)
5: (RPAREN , ')' , línea=1, col=12)
Errores léxicos :
Error léxico – línea 1, columna 6: identificador inválido '123abc' (los identificadores no pueden comenzar con un dígito)

Expresiones analizadas : 2
```

## 5.3 Identificadores Inválidos y/o símbolos desconocidos

Archivo de entrada – tests/invalid/errores02.txt

# errores02.txt – Entradas con símbolos desconocidos adicionales  
# Cada línea es una expresión independiente.

SUMA(A1 # B5)  
\$\$invalido

Salida y Ejecución

=====

Archivo : errores02.txt

=====

Línea 4: SUMA(A1 # B5)

Tokens reconocidos : 6

- 1: (FUNCTION , 'SUMA' , línea=1, col=1)
- 2: (LPAREN , '(' , línea=1, col=5)
- 3: (CELL\_REF , 'A1' , línea=1, col=6)
- 4: (ERROR , '#' , línea=1, col=9)
- 5: (CELL\_REF , 'B5' , línea=1, col=11)
- 6: (RPAREN , ')' , línea=1, col=13)

Errores léxicos :

Error léxico – línea 1, columna 9: símbolo desconocido '#'

Línea 5: \$\$invalido

Tokens reconocidos : 3

- 1: (ERROR , '\$' , línea=1, col=1)
- 2: (ERROR , '\$' , línea=1, col=2)
- 3: (IDENTIFIER , 'invalido' , línea=1, col=3)

Errores léxicos :

Error léxico – línea 1, columna 1: símbolo desconocido '\$'

Error léxico – línea 1, columna 2: símbolo desconocido '\$'

Expresiones analizadas : 2

## 6. Conclusiones

### Aprendizajes Individuales

Brandon. Desarrollar este proyecto me ayudo a tener una comprensión más amplia de lo aprendido en clases, así como del uso que se le puede dar a herramientas externas como ANTLR al momento de diseñar un lenguaje personalizado. Considero que hemos optado por una buena propuesta que me gustaría seguir trabajando en la continuación de la materia.

Celine. Con este proyecto pude entender mejor cómo funciona un analizador léxico y cómo aplicar lo aprendido en clase en algo más práctico, también me ayudó a familiarizarme con el uso de ANTLR4 y entre las demás mencionadas en clase, con la forma en que se pueden reconocer y traducir expresiones de Excel a Java.

Juan. Este proyecto me permitió ver de manera práctica cómo se construye la primera fase de un compilador. Entender que el lexer no evalúa, sino que clasifica fue clave para comprender el proceso completo de traducción. El uso de ANTLR4 me pareció una herramienta muy poderosa que facilita enormemente la definición de reglas léxicas y que definitivamente consideraría utilizar en proyectos futuros.

Carlos. Participar en este proyecto fue una experiencia que consolidó varios conceptos de la materia que en teoría me costaba visualizar. Ver cómo una fórmula de Excel se descompone en tokens con tipo, valor y posición, y entender los retos que eso implica, como la ambigüedad de ciertas palabras o el comportamiento de herramientas como ANTLR, me dejó con una perspectiva mucho más clara de lo que implica diseñar un lenguaje de programación desde cero.



## Dificultades encontradas durante el desarrollo

Una de las primeras confusiones surgió al querer delimitar con exactitud que le corresponde hacer al lexer, específicamente la presencia de referencias a celdas como A1 o rangos como A1:B5. La resolución de rangos en arreglos JAVA corresponderán a las fases de análisis semántico y generación de código.

Al reorganizar el proyecto a la estructura de carpetas requerida en las especificaciones del proyecto. El IDE mostraba el error *"Cannot find module ExcelLexer"* en `src/lexer.py` aunque el programa funcionaba adecuadamente en tiempos de ejecución. El problema se originó en que la manipulación dinámica de `sys.path` no puede ser resuelta por el analizador estático de Python (Pylance/Pyright). La solución requirió hacer de `grammar/` un paquete Python mediante `__init__.py`, agregar la raíz del proyecto al path (en lugar del subdirectorio `generated/`) y crear un archivo `pyrightconfig.json` que le indicara al IDE cuál era el directorio raíz de los módulos.

ANTLR4 aplica la regla *maximal munch*: al encontrar 123abc, consume 123 con la regla INTEGER y luego abc con la regla IDENTIFIER, produciendo dos tokens válidos sin lanzar ningún error. Esta conducta es correcta desde el punto de vista de ANTLR, pero incorrecta desde las reglas del lenguaje, donde un identificador no puede comenzar con un dígito. Fue necesario implementar un chequeo de adyacencia en la capa Python que recorre los tokens ANTLR crudos y detecta cuando un número y un identificador están en posiciones consecutivas sin espacio entre ellos, reportando el error manualmente.

Las palabras Y, O y NO presentan una ambigüedad léxica: pueden ser funciones lógicas ( $Y(A1 > 0, B1 < 10)$ ) o identificadores (Y como nombre de variable). En una gramática convencional esto requeriría contexto del parser. La solución implementada usa un predicado semántico de ANTLR directamente en el lexer: `{ self._input.LA(1) == ord('(') }?`, que verifica si el siguiente carácter en el stream es un paréntesis de apertura antes de clasificar el token como FUNCTION.

Al introducir caracteres Unicode decorativos en la salida (como = para los banners), el programa fallaba en terminales Windows con encoding cp1252. La solución fue agregar al inicio de `main.py` una reconfiguración explícita del stdout a UTF-8 mediante `sys.stdout.reconfigure(encoding='utf-8')`, únicamente cuando el encoding detectado no fuera ya UTF-8.



## Trabajo Futuro

El siguiente paso es construir un analizador sintáctico que consuma la lista de tokens producida por el lexer y valide que su estructura corresponde a expresiones matemáticas bien formadas

Una vez validada la estructura, el análisis semántico verificaría reglas de significado, como que los rangos sean coherentes (que A1:A3 no esté "al revés"), que los argumentos de cada función correspondan con el número esperado, y que los operadores se usen con tipos compatibles.

La fase final del traductor sería tomar el árbol sintáctico y generar el método Java equivalente. Un caso concreto: dado POTENCIA(A1, 2), el generador produciría:

```
public double calcular(double A1) {  
    return Math.pow(A1, 2);  
}
```

Para funciones con rangos, el traductor expandiría SUMA(A1:A3) en un método Java con los parámetros double A1, double A2, double A3 y el cuerpo return A1 + A2 + A3;

## Posibles Mejoras

El lexer actual reconoce 14 funciones. Una versión extendida podría incluir funciones financieras (VPN, TIR), de búsqueda (BUSCARV, BUSCARH) y estadísticas (DESVEST, VARIANZA),

Actualmente el programa se ejecuta desde la línea de comandos. Una mejora significativa sería desarrollar una interfaz gráfica (web o de escritorio) donde el usuario ingrese su fórmula Excel directamente y visualice el código Java generado lado a lado, haciendo la herramienta accesible para usuarios no técnicos.

## 7. Referencias

- ANTLR4 Documentation. Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.
- Python antlr4-python3-runtime v4.13.2. <https://pypi.org/project/antlr4-python3-runtime/>
- Microsoft Excel — Referencia de funciones: <https://support.microsoft.com/excel>

